

CXL RAS Architecture & Emulation Validation

1. Introduction & Master System Topology

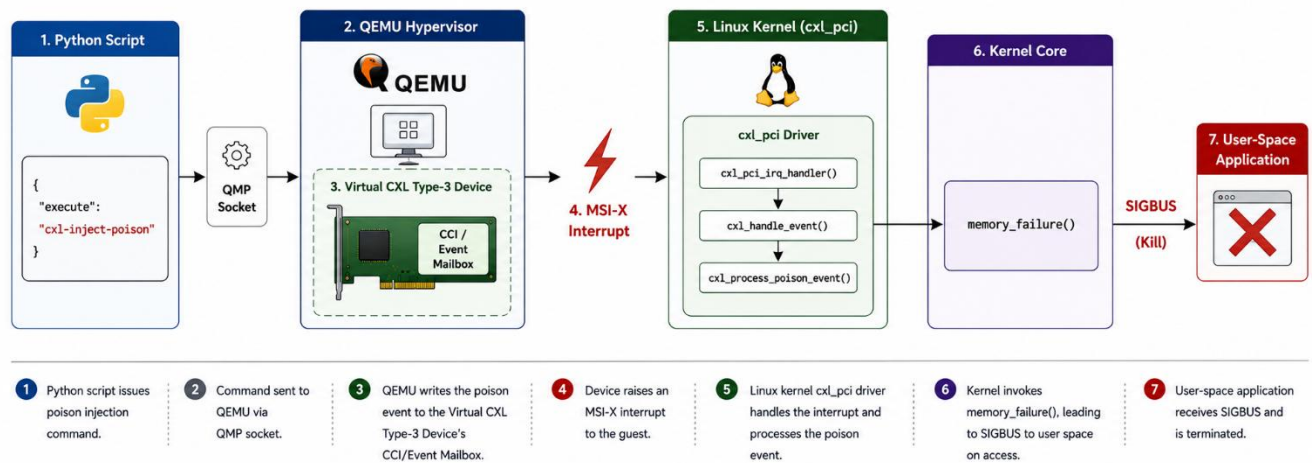
1.1 Abstract

As warehouse-scale data centers move toward composable infrastructure, Compute Express Link (CXL) has become a key technology for enabling pooled and shared memory architectures. However, connecting large volatile memory pools across multiple compute nodes also introduces significant reliability challenges. A single unmanaged hardware fault can quickly propagate across the system, potentially causing multi-tenant server crashes and Silent Data Corruption (SDC). Addressing these risks requires a shift away from traditional Firmware-First error handling approaches toward faster and more transparent OS-First recovery mechanisms.

This document presents a detailed architectural study of Reliability, Availability, and Serviceability (RAS) features in CXL 2.0 and 3.1. In addition, it describes the development of a fully software-defined validation framework built using QEMU-based emulation. By leveraging out-of-band QEMU Machine Protocol (QMP) sockets together with Linux kernel debugfs interfaces, the framework enables deterministic hardware fault injection, minimizes firmware-induced latency, and validates the Linux kernel's ability to isolate poisoned memory pages through `memory_failure()` without triggering system-wide failures.

1.2 Master Architecture Diagram

The validation stack runs from a Python test controller at the top through QEMU hypervisor emulation, a virtual CXL Type-3 device with a CCI mailbox, the MSI-X interrupt path into the Linux CXL driver, and finally the kernel memory failure subsystem. The diagram below illustrates this flow.



Each layer in this stack corresponds to a dedicated section in this document. The Python controller initiates fault injection via QMP. QEMU models the resulting hardware state change within the virtual CXL device . The cxl_pci driver reads the event mailbox following MSI-X delivery, translates the DPA to an HPA, generates a CPER record, and calls `memory_failure()` for page isolation.

2. Foundations of Enterprise RAS

2.1 Reliability, Availability, and Serviceability

Enterprise server infrastructure is evaluated across three dimensions collectively referred to as RAS.

- **Reliability:** The statistical probability that a hardware component will perform its required function without degradation over a specified time period. It is achieved through robust silicon design, Error-Correcting Code (ECC), and proactive media scrubbing.
- **Availability:** The mathematical calculation of system uptime. In enterprise clusters targeting "Five Nines" (99.999%) availability, the architecture cannot afford to crash an entire server due to a localized fault. The system must degrade gracefully.
- **Serviceability:** The capacity for infrastructure teams to diagnose, hot-swap, and repair degraded hardware without halting the overarching compute cluster. This requires pinpoint telemetry and out-of-band management protocols.

2.2 The Physics of Memory Degradation

Memory errors are not exceptional events. DRAM cells are susceptible to cosmic ray-induced soft errors, with error rates varying based on technology node and environmental conditions. Modern DDR4/DDR5 exhibits correctable error rates typically in the range of 1,000–10,000 FIT (Failures In Time per billion device-hours) under typical data center conditions. Beyond cosmic radiation, three silicon aging mechanisms progressively degrade memory reliability over multi-year operation: Hot Carrier Injection (HCI) shifts transistor threshold voltages; Negative Bias Temperature Instability (NBTI) causes PMOS threshold degradation under sustained voltage stress; and electromigration displaces material in metal interconnects under high current density. None of these mechanisms announce themselves — they accelerate gradually, and uncorrected single-bit errors accumulate until a second simultaneous error in the same ECC word produces an uncorrectable multi-bit fault.

The most dangerous outcome of an undetected memory error is Silent Data Corruption (SDC): a bit flip that bypasses ECC detection, gets written to persistent storage or used in an active computation, and produces incorrect results that go unnoticed until they propagate into databases, AI training weights, or financial records. Preventing SDC is the primary motivation for the precision telemetry and surgical isolation mechanisms that CXL 2.0 provides.

2.3 The Business Case for OS-First RAS

Legacy recovery strategies responded to uncorrectable memory errors with a global machine halt. In a direct-attached memory model, this was a contained failure: one server goes down, one workload restarts. CXL memory pooling breaks this containment. A CXL Type-3 device can be logically partitioned using Multi-Logical Device (MLD) mode to support up to 16 independent

Logical Device IDs (LD-IDs), each accessible by a separate host. Alternatively, in switched CXL topologies, multiple hosts can access a shared device through fabric-level routing. A naive global halt in response to a fault on that device crashes all 16 tenants, turning a localized silicon defect into a cluster-wide outage.

Advanced RAS infrastructure converts this scenario into a contained, recoverable event: the faulty page is offlined, the affected process receives a SIGBUS and terminates cleanly, and the remaining 15 tenants continue operating without interruption. Beyond immediate fault containment, the per-DRAM-chip error telemetry exposed through the CXL CCI mailbox enables predictive maintenance — identifying components whose correctable error rate is trending upward before they generate an uncorrectable fault, allowing scheduled replacement during planned maintenance windows rather than reactive emergency response.

3. CXL Architecture & Telemetry Evolution

3.1 The CXL Component Command Interface (CCI)

The CXL Component Command Interface (CCI) is a mailbox-based communication layer, defined in the CXL specification, through which the host OS and platform firmware exchange commands and event notifications with a CXL memory device. It operates over the CXL.io protocol layer, using the PCIe physical and link layers for transport, which maintains backward compatibility with existing PCIe infrastructure.

The mailbox is memory-mapped into the device’s Base Address Register (BAR0) space and is protected by a semaphore-based locking mechanism to prevent concurrent access from the host and device firmware. The host acquires the Mailbox Interface Ready semaphore, writes a command opcode and payload, polls the Background Command Status register for completion, then reads the response. The CCI command set includes Get Event Records (0x0100), Clear Event Records (0x0101), Get Poison List (0x4300), Clear Poison (0x4301), and a set of media management commands for PMEM devices. This richness is what distinguishes the CCI from legacy PCIe configuration space, which provides only coarse device-level error status with no mechanism to query specific fault addresses or media health state.

3.2 PCIe AER vs. CXL 2.0 Event Mailboxes

Because CXL is built on the PCIe electrical layer, it inherits the PCIe Advanced Error Reporting (AER) framework for link-layer faults such as framing errors, malformed TLPs, and receiver overflows. AER remains effective for those cases. However, it lacks the payload capacity to isolate memory errors down to specific addresses. When a memory cell fails under AER, the OS receives a generic interrupt indicating that a device on the bus encountered a fault; pinpointing the location requires either firmware intervention or accepting the uncertainty. CXL 2.0 Event Mailboxes address this limitation directly.

Metric	PCIe AER (Legacy)	CXL 2.0 Event Mailbox
Architectural scope	Physical link, I/O, and protocol errors	Coherent memory and cache data integrity
Fault granularity	Port-level or device-level (coarse)	Cacheline or page-level, with exact DPA payload
Address precision	Vague or absent	Exact Device Physical Address (DPA) + length
Interrupt delivery	Often routed through SMI / NMI (high latency)	Direct MSI-X memory write (low latency, ~100µs)
Out-of-band support	Limited; vendor-specific implementations	Native PLDM over MCTP; BMC-accessible via sideband
Error history	No persistent log; cleared on device reset	On-device event log with UUID tags; survives reset

Metric	PCIe AER (Legacy)	CXL 2.0 Event Mailbox
Noise suppression	No hardware-level rate limiting	Configurable CVME threshold via Leaky Bucket algorithm
OS recovery action	Device reset, FLR, or full system halt	Surgical page offlining via memory_failure()

4. Fault Taxonomy & Sub-OS Self-Healing

4.1 The Four-Tier Fault Severity Matrix

When the CXL memory controller detects an error, it classifies the fault into one of four severity tiers. The tier determines the escalation path, the urgency of host notification, and the extent of isolation required. The table below defines each tier along with its expected kernel behavior and the corresponding test case used in this validation.

Tier	Error Class	Characteristics & Trigger	Host Recovery Action
T1	Correctable Error (CE)	Single-bit ECC error corrected by hardware before data is consumed. No data loss. Event counted against CVME threshold. Interrupt fired only when threshold is crossed.	Log event; increment CVME counter. Page offlined only if CE rate persistently exceeds threshold. No process disruption.
T2	Uncorrectable Recoverable (UCR)	Multi-bit error exceeding ECC capability; data not yet consumed by an active thread. Hardware sets poison bit on the affected cacheline; DPA reported via mailbox.	MSI-X interrupt fires immediately. Driver reads DPA, translates to HPA, invokes memory_failure(). PG_hwpoison set; SIGBUS sent to affected process.
T3	Uncorrectable Non-Fatal (UNF)	Multi-bit corruption; data may have been consumed or affects device configuration space. Device remains operational after isolation.	Driver evaluates affected region. Attempts CCI Clear Poison on PMEM media. Affected region offlined; system continues with reduced capacity.
T4	Fatal Error	Catastrophic failure of the interconnect or memory media: total link loss, metadata corruption, or power rail failure. Device entirely compromised.	Machine Check Exception with MCA_PANIC flag. If kernel page tables are affected, kernel panic is unavoidable. Primary target of race condition testing.

4.2 Noise Reduction: CVME Thresholding and the Leaky Bucket Algorithm

A degrading CXL device may generate hundreds or thousands of Tier 1 correctable errors per hour. Forwarding each one as an independent MSI-X interrupt would consume CPU time in interrupt service routines and produce alert fatigue in monitoring systems, without providing any actionable information, a single CE is expected and self-corrected. The problem is not individual events but sustained elevated rates, which indicate physical degradation.

The CXL specification addresses this through the Correctable Volatile Memory Error (CVME) threshold and a hardware Leaky Bucket algorithm in the device firmware. The bucket counter increments on each new CE and decrements continuously at a configurable rate proportional to elapsed time. Only when the counter overflows the programmed threshold does the device fire

an MSI-X interrupt. This design filters transient error bursts caused by environmental conditions (temperature spikes, electromagnetic interference) from persistent degradation trends that require maintenance. When the OS receives a CVME threshold interrupt, it initiates an enhanced monitoring period and evaluates whether scheduled replacement is warranted.

4.3 Sub-OS Autonomous Healing

DDR5 Error Check and Scrub (ECS)

DDR5 modules include an on-module Error Check and Scrub engine that operates entirely within the DRAM device, below the visibility of the memory controller. The ECS engine periodically reads every memory cell, corrects single-bit errors via ECC, and writes back the corrected data. This proactive scrubbing prevents the accumulation of uncorrected single-bit errors that could, given a second simultaneous error in the same ECC word, escalate into an uncorrectable multi-bit fault. The scrub interval is configurable through the module's SPD registers via the DDR5 PMBus/I3C interface.

CXL 3.1 Dynamic Row Sparing

When the device's internal firmware detects that a DRAM row's correctable error rate is approaching the CVME threshold, it can autonomously migrate the data from that row to a pre-allocated spare row in the same bank group. The internal DPA-to-physical-row address translation tables are updated, and the completed sparing is reported to the host through an informational CCI mailbox entry (Event Class: General Media, Event Type: Memory Module Media Health Status Change). From the OS perspective, the DPA remains valid and accessible — the physical remapping is invisible above the CXL device layer. This mechanism can extend device lifetime significantly by redistributing write stress away from the most degraded rows.

4.4 The Discover-to-Verify Audit Trail

When autonomous repair occurs, the device must verify the replacement silicon before returning it to service. The CXL specification defines a structured workflow using Component UUIDs to track each fault from detection through verification:

- **Discover:** The device firmware assigns a UUID to the error event and stores it in the on-device event log. The host reads this log via CCI Get Event Records (opcode 0x0100) following an MSI-X interrupt.
- **Isolate:** The kernel translates the reported DPA to an HPA via HDM decoder tables, calls `memory_failure()`, and sets `PG_hwpoison` on the affected page. The UUID and DPA are recorded in the CPER log.
- **Remediate:** For PMEM, device firmware executes bad-block remapping. CCI Clear Poison (0x4301) is issued post-remediation to clear the hardware poison bit. For volatile DRAM, the page remains permanently offlined.
- **Verify:** The MARCH+ memory test algorithm — a deterministic read/write pattern designed to expose all single and double cell faults — is applied to the suspect region. Successful completion confirms the region is safe for reallocation.

5. System Software Handoff & Recovery Pipelines

5.1 The Legacy Model: Firmware-First via SMI

Traditional x86 server platforms intercept memory errors in firmware before the OS gains visibility. When the memory controller detects an uncorrectable error, it asserts a System Management Interrupt (SMI) — the highest-priority, non-maskable interrupt in the x86 architecture. All CPU cores immediately suspend OS execution and drop into System Management Mode (SMM), running from SMRAM, a protected memory region inaccessible to the OS.

UEFI firmware running in SMM reads the hardware error registers, attempts proprietary logging into the ACPI Boot Error Record Table (BERT), and eventually returns control to the OS. The performance cost of this model is significant: the SMI assertion, firmware processing, and SMM exit sequence together consume 100 microseconds to several milliseconds of wall-clock time during which every CPU core is completely frozen. For memory-bandwidth-intensive workloads or real-time systems, this latency is unacceptable. There is also an observability problem: the OS cannot see what the firmware did during SMM or evaluate the severity of the fault until control is returned.

5.2 The Hybrid Compromise

Some platforms use a hybrid model where firmware intercepts the fault first (via SMI) solely to log the event into BERT or HEST, then immediately asserts a System Control Interrupt (SCI) or NMI to notify the OS. This reduces the time firmware spends in SMM but does not eliminate the latency penalty entirely. The OS still receives the notification after firmware has processed it, and the actual recovery action — page offlining, process termination — is deferred to the OS. The hybrid model is a pragmatic compromise on platforms where pure OS-First support is not available in firmware.

5.3 The OS-First Pipeline and ACPI _OSC Negotiation

The OS-First model removes firmware from the error handling path entirely. Rather than an SMI, the CXL device fires an MSI-X interrupt — a memory write to a host-programmed address — that is delivered directly to the registered IRQ handler in the `cxl_pci` driver without pausing any CPU cores. This reduces error handling latency from tens of milliseconds to roughly 100 microseconds.

The transition to OS-First is not automatic. It requires an explicit capability negotiation at boot via the ACPI _OSC (Operating System Capabilities) control method. During ACPI enumeration, the Linux kernel invokes _OSC on the CXL host bridge, asserting a bitmask that claims native handling of CXL error events. If the platform firmware acknowledges the request by returning the same bitmask unmodified, it suppresses its own SMIs for CXL events from that point forward. The kernel then registers MSI-X interrupt handlers for all CXL devices via `pci_alloc_irq_vectors()` and `request_irq()`. If firmware does not grant the request — returning a modified bitmask with the claimed bits cleared — the system falls back to the hybrid or firmware-first model for that device.

5.4 DPA to HPA Address Translation

Once the OS-First pipeline is active and an MSI-X interrupt fires, the `cxl_pci` driver reads the event mailbox, which contains the Device Physical Address (DPA) of the corrupted memory. The DPA is the device's internal linear address space, ranging from 0 to the device's total capacity. The Linux memory manager operates in Host Physical Address (HPA) space — the system-wide physical address map enumerated from ACPI SRAT and the E820 memory map. These two address spaces are not directly related, and the mapping between them is not fixed.

The mapping is established at boot by the platform firmware, which programs the CXL device's Host-managed Device Memory (HDM) decoder registers. Each HDM decoder defines a tuple: (HPA base, size, interleave granularity, interleave ways, device position). The Linux CXL driver reads these decoder values during `cxl_pci_probe()` and stores them in the device's internal state structure. The translation process for an incoming DPA proceeds as follows:

- The driver iterates through the registered HDM decoders to find one whose DPA range covers the reported address.
- For a non-interleaved device: $HPA = HDM_base + DPA$.
- For an interleaved configuration: the driver applies the inverse interleave formula, accounting for the granularity and the device's position in the interleave set.
- The resulting HPA is passed to `memory_failure()`, which uses it to locate the affected struct page in the kernel's memory model and execute page isolation.

6. Advanced Containment & Media Integrity Edge Cases

6.1 Data Poisoning vs. Viral States

Two mechanisms control how far a memory error propagates through the CXL fabric. Data Poisoning is the standard, precise mechanism: when the device detects an uncorrectable error in a specific cacheline, it sets a hardware poison bit in the device's internal metadata for that DPA. Any subsequent read to that cacheline — by the host CPU, a peer CXL device, or a DMA engine — returns a poison completion value (a special PCIe TLP prefix). The consuming agent's memory controller detects this and signals an error, triggering the normal recovery path. The blast radius is exactly the affected 64-byte cacheline.

Viral State is a more aggressive mechanism used when the error cannot be contained at the cacheline level — for example, a routing failure in a CXL switch or internal metadata corruption in the memory controller. The device asserts the viral bit in all outgoing TLP prefixes, causing all connected agents to enter viral state as well, halting all I/O to and from the affected sub-fabric. Viral state guarantees that no corrupted data escapes the device but can affect multiple tenants sharing the same physical switch or host bridge.

6.2 Multi-Logical Device Viral Containment via LD-ID Vectors

A CXL Type-3 device operating in Multi-Logical Device (MLD) mode can be logically partitioned and assigned to multiple independent host CPUs. Each partition is identified by a Logical Device ID (LD-ID). If a global viral state were triggered on an MLD in response to a fault in one tenant's partition, it would crash every host mapped to the device, the same cluster-wide failure that CXL pooling was supposed to prevent.

CXL 2.0 and 3.1 address this by scoping the viral state assertion to the specific LD-ID where the containment breach occurred. The device firmware identifies the affected logical partition and isolates the viral condition to that LD-ID only. The one affected host receives a fatal error and halts; the remaining tenants continue operating on their respective partitions of the same physical device without interruption. This LD-ID vector mechanism is what makes fault isolation in shared memory environments viable at scale.

6.3 The Wear-Leveling Conflict in CXL Persistent Memory

The standard kernel recovery sequence — detect a fault, translate DPA to HPA, set PG_hwpoison, offline the page permanently — is designed for volatile DRAM where a faulty physical address represents a permanently failed cell. Applying this logic directly to CXL persistent memory (PMEM) backed by NAND flash or 3D XPoint creates an operational conflict.

PMEM devices use a Flash Translation Layer (FTL) that continuously remaps logical-to-physical block addresses to distribute write cycles evenly across the media, extending device lifespan. When the OS permanently offlines a DPA after a media error, it reduces the address space available to the FTL for future wear-leveling operations. This concentrates write traffic on the remaining cells and accelerates degradation. The correct approach for PMEM errors is to

quarantine the DPA temporarily, allow the device firmware to execute its internal bad-block remapping, verify the remapped region with MARCH+ patterns via CCI media management commands, then return the verified region to the allocatable pool. Standard kernel page offlining skips this recovery path entirely.

6.4 Catastrophic Power Loss: GPF and the Dirty Shutdown Count

PMEM devices often buffer incoming writes in volatile DRAM caches before flushing them to non-volatile media. A sudden power failure while the write buffer is non-empty leaves the device in an inconsistent state: some data is committed, some is lost, and the device's internal metadata structures (write journal, block allocation map) may be partially updated.

The CXL architecture handles this through the Global Persistent Flush (GPF) mechanism. The platform firmware programs a GPF sequence triggered by power-fail detection circuitry monitoring the bulk power supply rails. On voltage drop, GPF assertions are sent to all CXL PMEM devices, instructing them to drain their volatile write buffers to non-volatile media using energy stored in onboard supercapacitors — typically providing 5–50 ms of flush energy. The outcome is tracked by the Dirty Shutdown Count (DSC), a register stored in the device's non-volatile Label Storage Area (LSA). On the next power-up, the host driver compares its stored DSC against the device-reported DSC: a mismatch indicates the GPF did not complete, triggering journal replay and filesystem consistency checks before the PMEM namespace is mounted.

7. Emulation Architecture & Injection Mechanics (QEMU)

7.1 Why Software Emulation

Validating the OS-First recovery pipeline on physical CXL hardware presents a practical difficulty: triggering a deterministic, repeatable Tier 4 fatal error on real silicon typically requires hardware logic analyzers, fault injection interposers, or specialized test equipment not available in a development environment. QEMU eliminates this dependency. By virtualizing the PCIe bus, ACPI tables, and the physical memory backing store, QEMU allows the validation engineer to inject faults directly into the device's internal state out-of-band — without the guest OS's involvement — and observe the kernel's response in a controlled, repeatable manner.

The primary limitation of this approach is that QEMU's emulation is a faithful model of the CXL protocol state machine but not of physical hardware timing. Real MSI-X interrupt latency on physical CXL hardware is bounded by PCIe link-layer round-trip time (~100–200 ns); QEMU's virtual IRQ delivery adds scheduling overhead that varies with host CPU load. Physical DRAM timing parameters and real wear-leveling internals are not modeled. These constraints define the scope of the validation: what is being tested is the kernel driver's logical correctness, not its performance characteristics on physical hardware.

7.2 QEMU Topology Configuration

The virtual CXL environment requires QEMU's q35 machine type with CXL extensions enabled. The following command constructs a complete PCIe/CXL hierarchy: a CXL Host Bridge, a Root Port, and an attached Type-3 memory device backed by a host-side file.

The `-object memory-backend-file` entries provision host-side file-backed memory as the device's media and Label Storage Area (LSA). The `pxb-cxl` device emulates the CXL Host Bridge; `cxl-rp` emulates the Root Port; and `cxl-type3` is the Type-3 memory device. The `-M cxl-fmw` parameter configures the CXL Fixed Memory Window, which defines the HPA range mapped to the device address space. The QMP Unix socket is the fault injection entry point.

The guest kernel must be compiled with `CONFIG_CXL_BUS=y`, `CONFIG_CXL_MEM=y`, `CONFIG_MEMORY_FAILURE=y`, `CONFIG_ACPI_APEI_GHES=y`, and `CONFIG_PCIEAER_CXL=y`. Without these, the `cxl_pci` driver will not load and the `memory_failure()` routine will not be available.

```
scripts/config --enable CONFIG_CXL_BUS
scripts/config --enable CONFIG_CXL_PCI
scripts/config --enable CONFIG_CXL_MEM
scripts/config --enable CONFIG_CXL_PORT
```

```
scripts/config --enable CONFIG_CXL_ACPI
scripts/config --enable CONFIG_CXL_REGION
scripts/config --enable CONFIG_CXL_REGION_INVALIDATION_TEST
scripts/config --enable CONFIG_CXL_FEATURES
scripts/config --enable CONFIG_CXL_ATL
scripts/config --enable CONFIG_MEMORY_FAILURE
scripts/config --enable CONFIG_ACPI_APEI
scripts/config --enable CONFIG_ACPI_APEI_GHES
scripts/config --enable CONFIG_RAS
make olddefconfig
```

7.3 Out-of-Band Fault Injection via QMP

The QEMU Machine Protocol (QMP) is a JSON-RPC interface exposed over the Unix socket configured above. The Python controller connects to this socket at test initialization, negotiates capabilities, and issues fault injection commands.

When QEMU processes this command, it locates the memory-backend-file region corresponding to DPA 4096, sets the poison metadata bit in its internal memory model, writes an event entry to the virtual CCI event mailbox, and schedules an MSI-X interrupt delivery to the guest kernel via APICv virtualization. From the guest kernel's perspective, the interrupt is indistinguishable from a real hardware fault.

```
{"execute": "cxl-inject-dram-event", "arguments": {"path": "/machine/peripheral/cxl1", "type": 0, "flags": 0, "dpa": 4563402752, "dpa-length": 64}}
```

7.4 In-Band Observation via debugfs

The Linux kernel exposes CXL-specific diagnostic interfaces under `/sys/kernel/debug/cxl/`. These provide a complementary, kernel-side trigger path independent of QMP injection, used to verify that the driver's mailbox read path is functioning correctly:

- `inject_poison` — Writing a DPA hex value to this file forces the driver to issue a CCI Get Poison List command (0x4300) and process the returned records, exercising the same mailbox read path that MSI-X delivery triggers.

- `clear_poison` — Writing a DPA issues CCI Clear Poison (0x4301), instructing the device to remove the poison metadata for that address. Applicable to PMEM media with media-level clearing support.
- `poison` — A read-only file returning the aggregated poison list across all registered CXL devices, formatted as: `<device_serial> <dpa_hex> <length_bytes> <source_type>`

8. Kernel Source Code Execution Mapping

This section traces the complete kernel execution path from the initial MSI-X interrupt through to the SIGBUS signal sent to the affected process. Source file paths and function names reference Linux kernel v6.x.

8.1 Phase 1: Interrupt Intercept and Mailbox Scheduling

When QEMU delivers the virtual MSI-X interrupt to the guest kernel via APICv, it is dispatched to the handler registered during device probe. In `drivers/cxl/core/pci.c`, the `cxl_pci_probe()` function allocates MSI-X vectors via `pci_alloc_irq_vectors()` and registers the interrupt service routine via `request_irq()`.

The IRQ handler `cxl_irq_handler()` reads the Device Status register. On detecting the `EVENT_STATUS` bit — indicating a new entry in the event mailbox — it schedules a work item via `schedule_work()` rather than processing the mailbox directly. Mailbox reads involve polling loops incompatible with atomic interrupt context, so processing is deferred to the kernel's system workqueue.

8.2 Phase 2: Mailbox Read, Translation, and CPER Generation

The deferred work item calls `cxl_mem_get_poison()` in `drivers/cxl/mem.c`. This function constructs a CCI Get Poison List command (opcode 0x4300), writes it to the mailbox via `cxl_mbox_send_cmd()`, polls the Mailbox Status register, and reads the response payload — an array of CXL Poison Info structures, each specifying a DPA and length.

For each reported DPA, `cxl_dpa_to_hpa()` is called to produce the corresponding HPA via the HDM decoder table stored in the device's `cxl_dev_state` structure. The translated HPA is then passed to the memory failure handler. Simultaneously, if ACPI Platform Error Interfaces are active, `cper_print_cxl_mem_err()` in `drivers/firmware/efi/cper-cxl.c` formats the fault data into a Common Platform Error Record conforming to UEFI Appendix N. This record is written to the persistent APEI error storage via `erst_write()`, ensuring the fault history survives reboots.

8.3 Phase 3: `memory_failure()` and Page Isolation

The core isolation function is `memory_failure(pfn, flags)` in `mm/memory-failure.c`, where `pfn` is the Page Frame Number derived from the translated HPA. It executes the following sequence:

- **Page identification:** `pfn_to_page(pfn)` locates the corresponding struct page. A `PageHWPoison()` check guards against duplicate handling.

- **Lock acquisition:** The page lock and, for anonymous pages, the anon_vma lock are acquired to protect the reverse mapping tree.
- **Reverse mapping walk:** `rmap_walk()` traverses the complete reverse mapping tree (anon_vma chains for anonymous pages, address_space->i_mmap for file-backed pages) to identify every process and VMA mapped to the poisoned PFN.
- **Page poison flag:** PG_hwpoinson is set on the struct page via SetPageHWPoison(). The page's reference count is decremented.
- **Pool exclusion:** `__offline_isolated_pages()` removes the poisoned PFN from the buddy allocator's free lists. It will not be reallocated.

8.4 Phase 4: Process Notification via SIGBUS

For each process identified by `rmap_walk()`, the kernel calls `kill_procs()` to deliver a SIGBUS signal with a specific `si_code`:

- **BUS_MCEERR_AR (Action Required):** the process was actively accessing the poisoned page when the fault occurred. The thread is terminated immediately to prevent the corrupted data from being used further.
- **BUS_MCEERR_AO (Action Optional):** the poisoned page is mapped by the process but was not currently being accessed. The process receives the signal and may handle it gracefully — for example, by remapping a clean copy of the data — before exiting or continuing.

After all user-space consumers are notified and the page is removed from the allocatable pool, the recovery is complete. The system continues running; only the processes that held mappings to the affected page are affected. If the poisoned page is part of the kernel's own page tables or a critical subsystem data structure, `memory_failure()` triggers a kernel panic as a last resort.

9. Negative Testing & Race Condition Analysis

9.1 Testing Philosophy

Standard validation tests inject faults while the system is at rest and observe the recovery. That approach confirms that the happy path works but says nothing about driver behavior under concurrent events. In production, hardware faults do not wait for the system to be idle. They arrive during hot-plug operations, firmware updates, ACPI table refreshes, or power management transitions — exactly the moments when the driver has partially initialized data structures and not yet registered all of its synchronization primitives.

The negative testing phase in this validation deliberately targets these transition windows. The objective is not to break the kernel but to confirm that it handles concurrent failure correctly: aborting initialization cleanly, releasing all allocated structures, and leaving the system in a consistent state without deadlocks, use-after-free errors, or kernel panics.

9.2 The Vulnerability Window: ACPI Hot-Plug Initialization

Code inspection of `drivers/cxl/core/pci.c` identified a window during CXL device hot-plug where the driver is in a partially initialized state. When a CXL device is hot-plugged, the ACPI subsystem triggers `cxl_pci_probe()`. This function allocates the `cxl_dev_state` structure via `cxl_dev_state_create()` and begins HDM decoder table construction and `_OSC` negotiation. However, `request_irq()` — which registers the MSI-X interrupt handler for the new device — is called later in the initialization sequence.

Between `cxl_dev_state_create()` returning and `request_irq()` completing, the device structure exists but the interrupt handler is not registered. If a Tier 4 fatal error arrives during this window, the interrupt dispatch machinery may attempt to access uninitialized fields in the device structure, potentially dereferencing a null pointer or encountering a mutex that has not yet been initialized.

9.3 The Race Condition Attack

The race condition test uses a Python multi-threaded harness. Thread 1 triggers the ACPI hot-plug event via QMP. Thread 2 monitors the guest kernel's serial console output for the specific `dmesg` message emitted at the start of `_OSC` negotiation and immediately fires a Tier 4 fatal error when it appears.

9.4 Validation Criteria

The validation methodology defines five safety criteria that must all pass for safe concurrent fault handling:

- The kernel logs the fatal CXL error event via the `cxl_pci` driver (visible in `dmesg`).

- The hot-plug sequence is aborted — `cxl_pci_probe()` returns a non-zero error code and the device does not appear under `/sys/bus/cxl/devices/`.
- No `WARN_ON()`, `BUG_ON()`, or RCU stall warnings are emitted during or after the injection window.
- The system does not enter kernel panic — the serial console remains responsive and the guest kernel continues executing normally.
- No memory leaks are reported by KASAN (enabled via `CONFIG_KASAN=y`) — specifically, no `kmalloc`-allocated `cxl_dev_state` regions remaining allocated after probe failure.